

```
<style>
body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif; line-height: 1.6; padding: 2em; }
h1, h2, h3 { color: #1e293b; padding-bottom: 0.3em; }
h2 { border-bottom: 1px solid #e2e8f0; }
img { max-width: 100%; border-radius: 12px; }
table { border-collapse: collapse; width: 100%; }
th, td { border: 1px solid #cbd5e1; padding: 8px; }
code { background-color: #f1f5f9; padding: 2px 4px; border-radius: 4px; font-family: monospace; }
pre { background-color: #f0f1f2a; color: #8f9c; padding: 1em; border-radius: 8px; overflow-x: auto; }
</style>

<div align="center">

<h1> OBEMIC: NBA Accreditation & OBE Engine</h1>
<h3>Outcome-Based Education Management Information & Calculation</h3>
<p><i>A complete, automated software solution for Engineering Institutions in India to achieve NBA / ABET Accreditation without the manual paperwork</i></p>
</div>
```

NEXT.JS	14+	https://nextjs.org/
NODE.JS	20.0+	https://nodejs.org/
TYPESCRIPT	5.5	https://www.typescriptlang.org/
POSTGRESQL	16	https://www.postgresql.org/
PRISMA	7.9.1	https://www.prisma.io/

 Table of Contents

- [Project Overview for Stakeholders](#)
- [The Three User Portals](#)
- [How Attainment is Calculated \(The Mathematics\)](#)
- [Frontend Architecture & User Experience](#)
- [Backend Architecture: "Under the Hood"](#)
- [Database Schema & Structure](#)
- [Directory Structure](#)
- [Local Developer Setup Guide](#)

1. Project Overview for Stakeholders

Colleges and Universities seeking accreditation from the **National Board of Accreditation (NBA)** or **AICTE** are required to implement Outcome-Based Education (OBE). This means moving away from traditional grade-based marking and instead proving that students have successfully acquired specific skills and knowledge.

OBEMIC eliminates the need for faculty to manually maintain hundreds of complex Excel sheets. The system automatically calculates attainment levels, generates the required CO-PO mapping matrices, and produces formatted reports ready for the NBA committee's review.

Key OBE Terminology Used in the System:

- PEO (Program Educational Objectives):** What graduates are expected to achieve 3 to 5 years after graduation.
- PO (Program Outcomes):** The 12 standard engineering attributes defined by the NBA (e.g., Engineering Knowledge, Problem Analysis, Ethics).
- PSO (Program Specific Outcomes):** 2 to 4 outcomes specific to the department (e.g., specific to CSE, ECE, or Mechanical).
- CO (Course Outcomes):** 5 to 6 specific learning objectives for a particular subject/course.

2. The Three User Portals

OBEMIC is designed with three separate interfaces to ensure that Management, Faculty, and Students only see what they need to see.

A. The Admin Control Center (For HODs & NBA Coordinators)

Accessed via `/admin`. Only authorized administrators and Heads of Departments can enter.

- Academic Setup:** Easily create Academic Years, Semesters, and Departments.
- Subject Management:** Define the Course Outcomes (COs) for every subject and map them to the 12 Program Outcomes (CO-PO Matrix).
- Workload Allocation:** Assign faculty members to handle specific subjects and student batches/sections.
- Survey Management:** Open and close the "Course End Surveys" for students at the end of the semester.

B. The Faculty Dashboard (For Teaching Staff)

Accessed via `/faculty`. Designed to save hours of faculty time.

- My Subjects:** Faculty will only see the subjects that the HOD has assigned to them.
- Excel Upload Engine:** Faculty simply upload their standard mark sheets (Internal Mid-exams, University Externals, and Lab marks) in Excel format. The system instantly reads the marks and calculates the attainment automatically.
- Visual Dashboards:** Instantly displays bar charts showing how well the class performed in each Course Outcome.
- One-Click NBA Reports:** Generates a properly formatted, white-background PDF report containing all CO-PO calculations, ready to be printed and signed for the NBA file.

C. Student Portal (For Course End Surveys)

Accessed via `/survey`.

- Simple Login:** Students log in securely using their University Roll Number.
- Pending Surveys:** Students can see a list of subjects they need to provide feedback for.
- Feedback Form:** A mobile-friendly screen where students rate their confidence in learning each Course Outcome on a scale of 1 to 5 (Poor to Excellent). This data is automatically sent to the Faculty Dashboard for "Indirect Attainment" calculation.

3. How Attainment is Calculated (The Mathematics)

OBEMIC strictly follows the mathematical models recommended by the NBA. The calculations are processed instantly by the backend system.

A. Direct Attainment (Based on University & Mid Marks)

This measures actual academic performance through exams.

- The Target (Threshold):** For example, the department sets a target that students should score $\geq 60\%$ of the maximum marks.
- Attainment Level (Linear Scale):**
Instead of rigid tiers, OBEMIC uses a precise **Direct Linear Proportional Scale** to be as fair and accurate as possible.
$$\text{Attainment Level (Out of 3)} = \frac{(\text{Pass Percentage} - \text{Threshold})}{(\text{Max Marks} - \text{Threshold})} \times 3$$

(Example: If 40% of the class passed the threshold, the attainment level is exactly 1.20)

B. Indirect Attainment (Based on Student Surveys)

This is calculated automatically from the Student Portal. The system takes the average of all 5-point ratings submitted by the students and scientifically scales it down to a standard 3-point level.

C. Final CO Attainment

Both direct (exams) and indirect (surveys) results are combined. Typically, exam marks carry more weight:

$$\text{Final CO Attainment} = (80\% \text{ of Exam Score}) + (20\% \text{ of Survey Score})$$

D. PO Attainment (The Final Goal)

To find out how much a specific subject contributed to the overall engineering degree (Program Outcomes), the system checks the Correlation Factor set by the HOD (1 = Low, 2 = Medium, 3 = High) and calculates:

$$\text{PO Attainment} = \text{Average of (Final CO Score} \times \text{Correlation Factor)}$$

4. Frontend Architecture & User Experience

The visual interface is built using modern technologies to ensure it is fast, responsive, and easy for non-technical staff to use.

Technical Stack

- Framework:** Next.js 14+ (React)
- Styling:** Tailwind CSS for a clean, professional, and accessible design.
- Charts:** Chart.js for rendering dynamic attainment graphs.

Key Features for the End-User

- No Page Reloads:** The system uses React State Management, meaning navigating between subjects and uploading marks happens instantly without the web page needing to reload.
- Print-Optimized Reports:** When a faculty member clicks "Print", the system uses advanced CSS `@media print` rules. It hides the menus, removes the darkmode colors, forces an A4 page width, and ensures the graphs are perfectly sized for a physical printer.
- Secure Sessions:** If a faculty or admin leaves their desk and their session expires, the system automatically redirects them to the login screen to protect student data.

5. Backend Architecture: "Under the Hood"

The backend is completely separated from the frontend, ensuring high security and performance when processing large Excel files.



The 5 Layers of the Backend

- 1. **Routes:** Manages the URLs (e.g., /api/faculty/upload).
- 2. **Middleware:** Acts as a security guard. It ensures that students cannot access faculty routes, and verifies passwords.
- 3. **Controllers:** Receives the request from the user and hands it to the Service layer.
- 4. **Services (The Brain):** This is where the heavy lifting happens. It opens the uploaded Excel files, reads the student marks row-by-row, applies the NBA math formulas, and generates the final results.
- 5. **Repositories:** Safely saves the final calculated data into the PostgreSQL database.

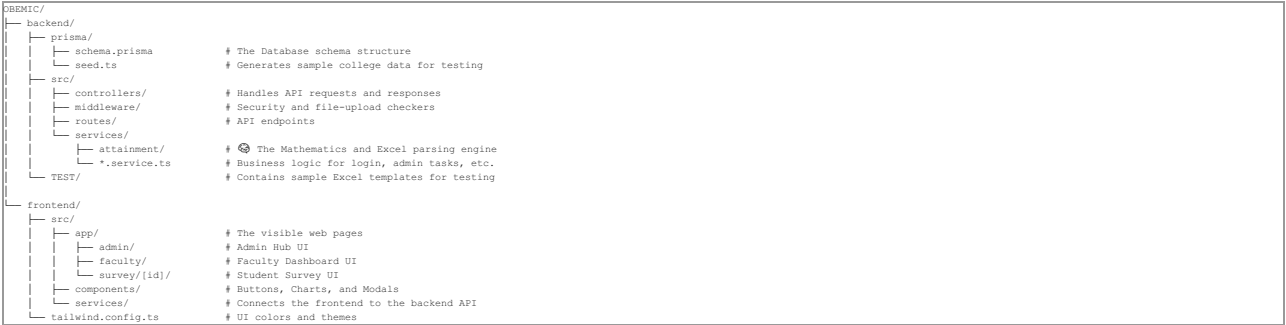
6. Database Schema & Structure

Powered by PostgreSQL and Prisma ORM, the database is structured to mirror an Indian engineering college's hierarchy.

- **Hierarchy:** Academic Year: $\rightarrow$ Semester $\rightarrow$ Subject $\rightarrow$ Course Outcome.
- **Faculty Assignment:** The most important table. It links a Faculty Member to a specific Subject and Batch/Section for a given year.
- **Reporting:** Stores the finalized attainment calculations so they can be reviewed and approved by the HOD.
- **Surveys:** Ensures that a student can only submit a survey for a subject exactly once.

7. Directory Structure

For the IT team, here is how the source code is organized:



8. Local Developer Setup Guide

If your IT team needs to set up the software on a local machine for testing or further development, follow these steps.

Prerequisites

- **Node.js:** v20+ recommended.
- **PostgreSQL:** Running locally on port 5432.

1. Database Configuration (Backend)

Inside the backend/ folder, create a .env file and add your database credentials:

```
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/obemic"
JWT_SECRET="your-secure-secret-key"
PORT=5000
```

2. Install & Prepare the Database

```
cd backend
npm install
npx prisma db push --force-reset
npx prisma generate
```

3. Generate Sample Data

Run this command to automatically create sample academic years, departments, an HOD, a faculty member, and students:

```
npx ts-node prisma/seed.ts
```

4. Start the Application

Start the Backend Server:

```
cd backend
npm run dev
```

Start the Frontend Website:

Open a new terminal window.

```
cd frontend
npm install
npm run dev
```

5. Access the System

- **Admin Hub:** Go to <http://localhost:3000/admin> (Login with `admin@college.edu` / `password123`)
- **Faculty Dashboard:** Go to <http://localhost:3000/faculty>
- **Student Survey Portal:** Go to <http://localhost:3000/survey>

Welcome to OBEMIC. 🧑🏻